

SKILL WEEK-4 : TRIGGERS AND PROCEDURES

2520080061

E.Pranay Reddy

11. PROCEDURE TO DISPLAY ALL CUSTOMERS

```
DELIMITER //
CREATE PROCEDURE GetAllCustomers()
BEGIN
    SELECT * FROM Customer;
END //
DELIMITER ;
```

CALL GetAllCustomers();

The screenshot shows a database management tool interface. On the left, there's a sidebar with 'Administrative' and 'Information' tabs. The main area displays a SQL script with line numbers 521 to 529. The script defines a procedure 'GetAllCustomers()' that selects all data from the 'Customer' table. Below the script, a 'Result Grid' shows the output of the procedure call. The grid has columns for Customer_ID, Customer_Name, Phone, Email, and City. It contains 10 rows of customer data.

Customer_ID	Customer_Name	Phone	Email	City
101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
104	Sneha Rao	9876543213	sneha@gmail.com	Chennai
105	Kiran Kumar	9876543214	kiran@gmail.com	Hyderabad
106	Anil Kumar	9876543215	anil@gmail.com	Delhi
107	Meena Reddy	9876543216	meena@gmail.com	Mumbai
108	Rahul Sharma	9876543217	rahul@gmail.com	Pune
109	Lakshmi Devi	9876543218	lakshmi@gmail.com	Hyderabad
110	Suresh Babu	9876543219	suresh@gmail.com	Vijayawada

12. PROCEDURE TO DISPLAY ACCOUNT DETAILS FOR A GIVEN ACCOUNT NUMBER

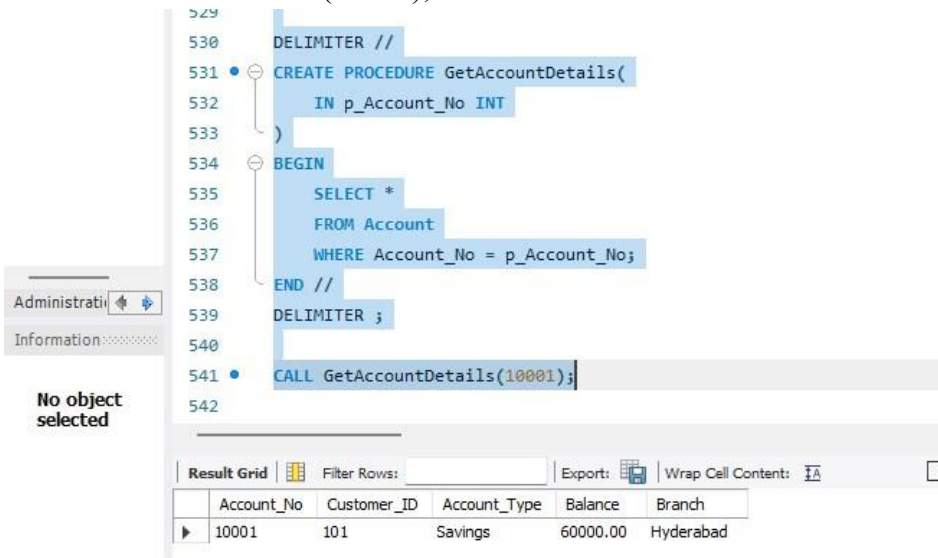
```
DELIMITER //
CREATE PROCEDURE GetAccountDetails(
    IN p_Account_No INT
)
```

```

BEGIN
    SELECT *
    FROM Account
    WHERE Account_No = p_Account_No;
END //
DELIMITER ;

```

```
CALL GetAccountDetails(10001);
```



The screenshot shows a SQL IDE with a script editor on the left and a result grid on the right. The script editor contains the following SQL code:

```

529
530 DELIMITER //
531 CREATE PROCEDURE GetAccountDetails(
532     IN p_Account_No INT
533 )
534 BEGIN
535     SELECT *
536     FROM Account
537     WHERE Account_No = p_Account_No;
538 END //
539 DELIMITER ;
540
541 CALL GetAccountDetails(10001);
542

```

The result grid shows the output of the procedure call:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	60000.00	Hyderabad

13. PROCEDURE TO FIND CUSTOMER ACCOUNTS

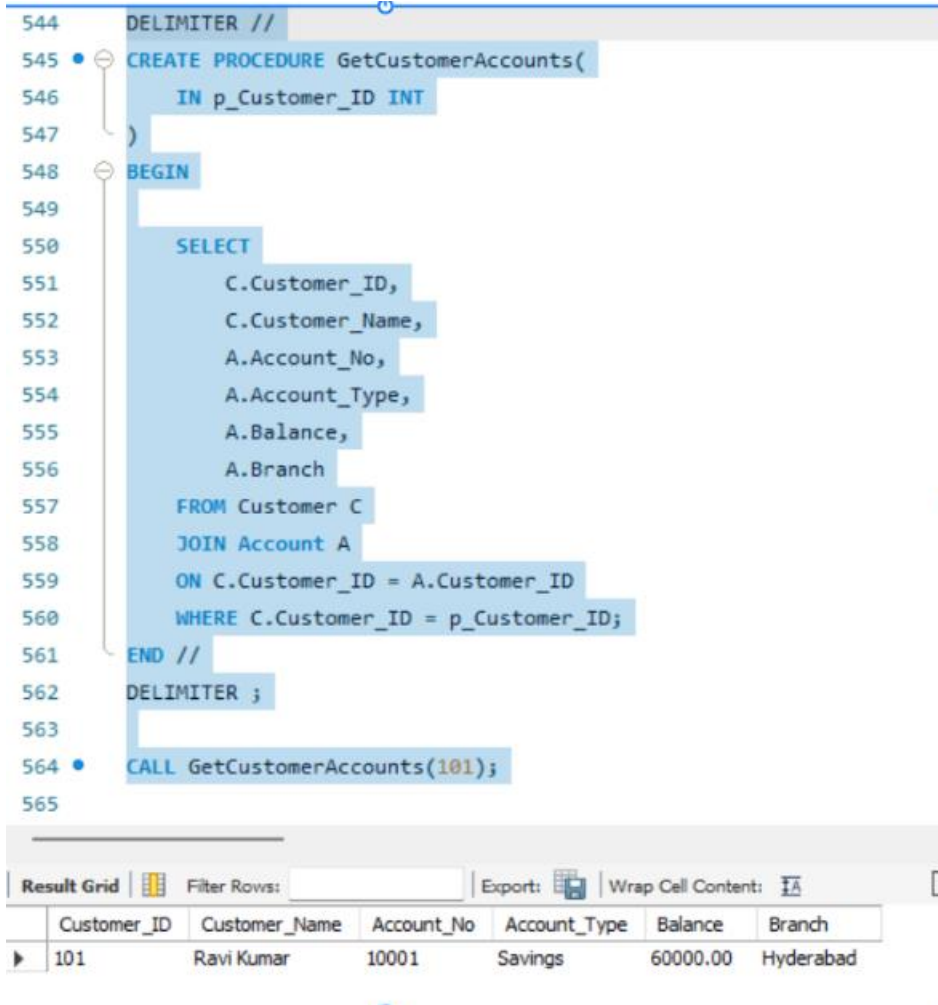
```

DELIMITER //
CREATE PROCEDURE GetCustomerAccounts(
    IN p_Customer_ID INT
)
BEGIN
    SELECT
        C.Customer_ID,
        C.Customer_Name,
        A.Account_No,
        A.Account_Type,
        A.Balance,
        A.Branch
    FROM Customer C
    JOIN Account A
    ON C.Customer_ID = A.Customer_ID

```

```
WHERE C.Customer_ID = p_Customer_ID;
END //
DELIMITER ;
```

```
CALL GetCustomerAccounts(101);
```



The screenshot displays a SQL IDE interface. The main editor window shows a SQL script with line numbers 544 through 565. The script defines a stored procedure named `GetCustomerAccounts` that takes an integer parameter `p_Customer_ID`. The procedure's body consists of a `SELECT` statement that joins the `Customer` table (`C`) with the `Account` table (`A`) on the condition `C.Customer_ID = A.Customer_ID`. The `SELECT` statement retrieves the following columns: `C.Customer_ID`, `C.Customer_Name`, `A.Account_No`, `A.Account_Type`, `A.Balance`, and `A.Branch`. The procedure concludes with `END //` and `DELIMITER ;`. Below the script, the `CALL GetCustomerAccounts(101);` statement is executed. At the bottom of the IDE, a 'Result Grid' is visible, showing a single row of data corresponding to the execution of the procedure with `Customer_ID` 101.

```
544 DELIMITER //
545 CREATE PROCEDURE GetCustomerAccounts(
546     IN p_Customer_ID INT
547 )
548 BEGIN
549
550     SELECT
551         C.Customer_ID,
552         C.Customer_Name,
553         A.Account_No,
554         A.Account_Type,
555         A.Balance,
556         A.Branch
557     FROM Customer C
558     JOIN Account A
559     ON C.Customer_ID = A.Customer_ID
560     WHERE C.Customer_ID = p_Customer_ID;
561 END //
562 DELIMITER ;
563
564 CALL GetCustomerAccounts(101);
565
```

Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
101	Ravi Kumar	10001	Savings	60000.00	Hyderabad

14. PROCEDURE TO DEPOSIT MONEY

```
DELIMITER //
CREATE PROCEDURE DepositMoney(
    IN p_Account_No INT,
    IN p_Amount DECIMAL(12,2)
)
BEGIN
    UPDATE Account
    SET Balance = Balance + p_Amount
    WHERE Account_No = p_Account_No;
```

```
END //
DELIMITER ;
```

```
CALL DepositMoney(10001, 5000);
```

```
566 DELIMITER //
567 • CREATE PROCEDURE DepositMoney(
568     IN p_Account_No INT,
569     IN p_Amount DECIMAL(12,2)
570 )
571 BEGIN
572     UPDATE Account
573     SET Balance = Balance + p_Amount
574     WHERE Account_No = p_Account_No;
575 END //
576 DELIMITER ;
577
578 • CALL DepositMoney(10001, 5000);
579
580
```

Output

#	Time	Action	Message
✓ 59	20:40:48	CALL GetAccountDetails(10001)	1 row(s) returned
✓ 60	20:41:24	CREATE PROCEDURE GetCustomerAccounts(IN p_Cu...	0 row(s) affected
✓ 61	20:41:24	CALL GetCustomerAccounts(101)	1 row(s) returned
✗ 62	20:41:30	CREATE PROCEDURE GetCustomerAccounts(IN p_Cu...	Error Code: 1304. PROCEDURE Gett
✗ 63	20:41:32	CREATE PROCEDURE GetCustomerAccounts(IN p_Cu...	Error Code: 1304. PROCEDURE Gett
✓ 64	20:41:43	CALL GetCustomerAccounts(101)	1 row(s) returned
✓ 65	20:42:26	CREATE PROCEDURE DepositMoney(IN p_Account_...	0 row(s) affected
✓ 66	20:42:26	CALL DepositMoney(10001, 5000)	1 row(s) affected
✓ 67	20:42:34	CALL DepositMoney(10001, 5000)	1 row(s) affected

15. PROCEDURE TO WITHDRAW MONEY

```
DELIMITER //
CREATE PROCEDURE WithdrawMoney(
    IN p_Account_No INT,
    IN p_Amount DECIMAL(12,2)
)
BEGIN
    UPDATE Account
    SET Balance = Balance - p_Amount
    WHERE Account_No = p_Account_No;
END //
DELIMITER ;
```

```
CALL WithdrawMoney(10001, 3000);
```

```

580 DELIMITER //
581 CREATE PROCEDURE WithdrawMoney(
582     IN p_Account_No INT,
583     IN p_Amount DECIMAL(12,2)
584 )
585 BEGIN
586     UPDATE Account
587     SET Balance = Balance - p_Amount
588     WHERE Account_No = p_Account_No;
589 END //
590 DELIMITER ;
591
592 CALL WithdrawMoney(10001, 3000);
593

```

Output

#	Time	Action	Message
61	20:41:24	CALL GetCustomerAccounts(101)	1 row(s) returned
62	20:41:30	CREATE PROCEDURE GetCustomerAccounts(IN p_Cu...	Error Code: 1304. PROCEDURE (
63	20:41:32	CREATE PROCEDURE GetCustomerAccounts(IN p_Cu...	Error Code: 1304. PROCEDURE (
64	20:41:43	CALL GetCustomerAccounts(101)	1 row(s) returned
65	20:42:26	CREATE PROCEDURE DepositMoney(IN p_Account_...	0 row(s) affected
66	20:42:26	CALL DepositMoney(10001, 5000)	1 row(s) affected
67	20:42:34	CALL DepositMoney(10001, 5000)	1 row(s) affected
68	20:43:19	CREATE PROCEDURE WithdrawMoney(IN p_Account_...	0 row(s) affected
69	20:43:19	CALL WithdrawMoney(10001, 3000)	1 row(s) affected

TRIGGERS

16. TRIGGER – PREVENT NEGATIVE BALANCE

```

DELIMITER //
CREATE TRIGGER CheckBalance
BEFORE UPDATE ON Account
FOR EACH ROW
BEGIN
    IF NEW.Balance < 0 THEN

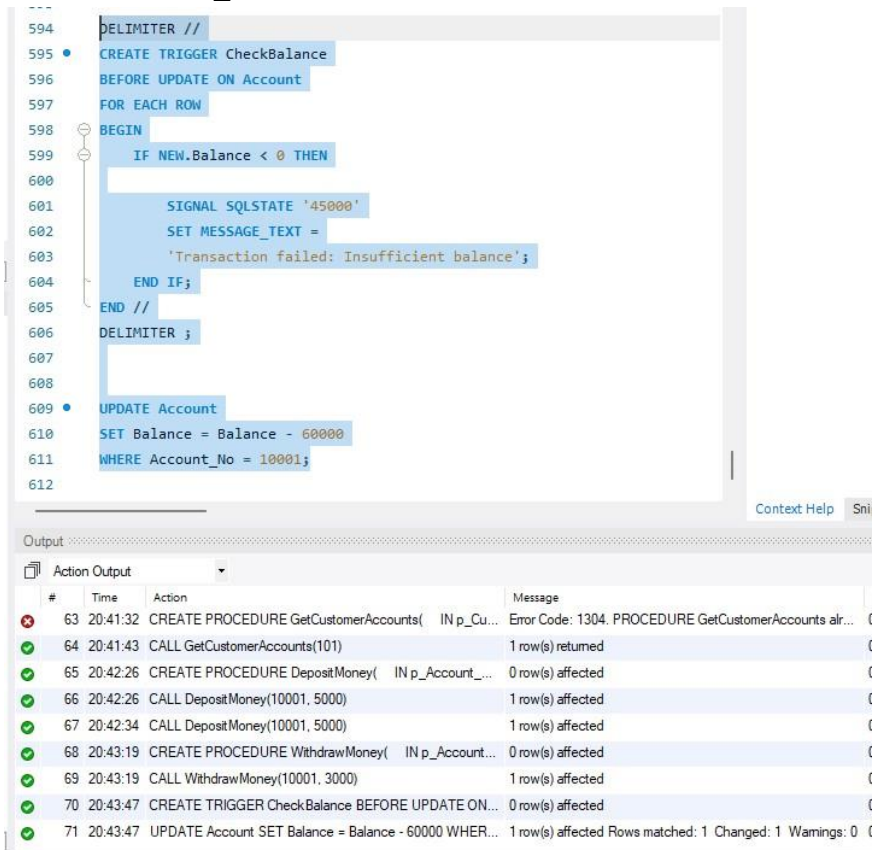
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
            'Transaction failed: Insufficient balance';
    END IF;
END //
DELIMITER ;

```

```

UPDATE Account
SET Balance = Balance - 60000
WHERE Account_No = 10001;

```



```

594 DELIMITER //
595 CREATE TRIGGER CheckBalance
596 BEFORE UPDATE ON Account
597 FOR EACH ROW
598 BEGIN
599     IF NEW.Balance < 0 THEN
600
601         SIGNAL SQLSTATE '45000'
602         SET MESSAGE_TEXT =
603         'Transaction failed: Insufficient balance';
604     END IF;
605 END //
606 DELIMITER ;
607
608
609 UPDATE Account
610 SET Balance = Balance - 60000
611 WHERE Account_No = 10001;
612

```

#	Time	Action	Message
63	20:41:32	CREATE PROCEDURE GetCustomerAccounts(IN p_Cu...	Error Code: 1304. PROCEDURE GetCustomerAccounts alr...
64	20:41:43	CALL GetCustomerAccounts(101)	1 row(s) returned
65	20:42:26	CREATE PROCEDURE DepositMoney(IN p_Account_...	0 row(s) affected
66	20:42:26	CALL DepositMoney(10001, 5000)	1 row(s) affected
67	20:42:34	CALL DepositMoney(10001, 5000)	1 row(s) affected
68	20:43:19	CREATE PROCEDURE WithdrawMoney(IN p_Account...	0 row(s) affected
69	20:43:19	CALL WithdrawMoney(10001, 3000)	1 row(s) affected
70	20:43:47	CREATE TRIGGER CheckBalance BEFORE UPDATE ON...	0 row(s) affected
71	20:43:47	UPDATE Account SET Balance = Balance - 60000 WHER...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

17. TRIGGER – PREVENT NEGATIVE DEPOSIT

```

DELIMITER //
CREATE TRIGGER CheckTransactionAmount
BEFORE INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    IF NEW.Amount <= 0 THEN

        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Transaction amount must be greater than zero';

    END IF;

```

```
END //
DELIMITER ;
```

```
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', -5000);
```

The screenshot displays a SQL IDE with a script editor and an output window. The script editor contains the following SQL code:

```
613 DELIMITER //
614 CREATE TRIGGER CheckTransactionAmount
615 BEFORE INSERT ON Bank_Transaction
616 FOR EACH ROW
617 BEGIN
618     IF NEW.Amount <= 0 THEN
619         SIGNAL SQLSTATE '45000'
620         SET MESSAGE_TEXT =
621             'Transaction amount must be greater than zero';
622     END IF;
623 END //
624 DELIMITER ;
625
626 INSERT INTO Bank_Transaction
627 (Account_No, Transaction_Type, Amount)
628 VALUES
629 (10001, 'DEPOSIT', -5000);
630
631
632
```

The output window shows the execution results of the script. It includes a table with columns: #, Time, Action, Message, and Duration. The table contains 13 rows of data, including the execution of the trigger and the insertion of a row into the Bank_Transaction table. The last row (73) shows an error: "Error Code: 1644, Transaction amount must be greater than..."

#	Time	Action	Message	Duration
65	20:42:26	CREATE PROCEDURE DepositMoney(IN p_Account_...	0 row(s) affected	0.0
66	20:42:26	CALL DepositMoney(10001, 5000)	1 row(s) affected	0.0
67	20:42:34	CALL DepositMoney(10001, 5000)	1 row(s) affected	0.0
68	20:43:19	CREATE PROCEDURE WithdrawMoney(IN p_Account_...	0 row(s) affected	0.0
69	20:43:19	CALL WithdrawMoney(10001, 3000)	1 row(s) affected	0.0
70	20:43:47	CREATE TRIGGER CheckBalance BEFORE UPDATE ON...	0 row(s) affected	0.0
71	20:43:47	UPDATE Account SET Balance = Balance - 60000 WHER...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.0
72	20:44:25	CREATE TRIGGER Check TransactionAmount BEFORE IN...	0 row(s) affected	0.0
73	20:44:25	INSERT INTO Bank_Transaction (Account_No, Transac...	Error Code: 1644, Transaction amount must be greater than...	0.0

18. CREATE TRANSACTION AUDIT TABLE

```
CREATE TABLE Transaction_Audit (
    Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
    Transaction_ID INT,
    Account_No INT,
    Transaction_Type VARCHAR(20),
    Amount DECIMAL(12,2),
    Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```

634 CREATE TABLE Transaction_Audit (
635     Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
636     Transaction_ID INT,
637     Account_No INT,
638     Transaction_Type VARCHAR(20),
639     Amount DECIMAL(12,2),
640     Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
641 );
642

```

Output

Action Output

#	Time	Action	Message
66	20:42:26	CALL DepositMoney(10001, 5000)	1 row(s) affected
67	20:42:34	CALL DepositMoney(10001, 5000)	1 row(s) affected
68	20:43:19	CREATE PROCEDURE WithdrawMoney(IN p_Account...	0 row(s) affected
69	20:43:19	CALL WithdrawMoney(10001, 3000)	1 row(s) affected
70	20:43:47	CREATE TRIGGER CheckBalance BEFORE UPDATE ON...	0 row(s) affected
71	20:43:47	UPDATE Account SET Balance = Balance - 60000 WHER...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
72	20:44:25	CREATE TRIGGER CheckTransactionAmount BEFORE IN...	0 row(s) affected
73	20:44:25	INSERT INTO Bank_Transaction (Account_No, Transactio...	Error Code: 1644. Transaction amount must be greater than...
74	20:45:09	CREATE TABLE Transaction_Audit (Audit_ID INT PRI...	0 row(s) affected

19. TRIGGER – TRANSACTION AUDIT

DELIMITER //

```

CREATE TRIGGER TransactionAudit
AFTER INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN

```

```

    INSERT INTO Transaction_Audit
    (
        Transaction_ID,
        Account_No,
        Transaction_Type,
        Amount
    )
    VALUES
    (
        NEW.Transaction_ID,
        NEW.Account_No,
        NEW.Transaction_Type,
        NEW.Amount
    );

```

```

END //
DELIMITER ;

INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 2500);

SELECT * FROM Transaction_Audit;

```

The screenshot shows a database management tool interface. The top pane displays SQL code being executed. The middle pane shows the 'Result Grid' with one row of data. The bottom pane shows the 'Action Output' log with a list of actions and their results.

SQL Code:

```

665 DELIMITER ;
666
667 INSERT INTO Bank_Transaction
668 (Account_No, Transaction_Type, Amount)
669 VALUES
670 (10001, 'DEPOSIT', 2500);
671
672 SELECT * FROM Transaction_Audit;
673

```

Result Grid:

Audit_ID	Transaction_ID	Account_No	Transaction_Type	Amount	Audit_Date
1	12	10001	DEPOSIT	2500.00	2026-08-13 20:45:40

Action Output:

#	Time	Action	Message
69	20:43:19	CALL WithdrawMoney(10001, 3000)	1 row(s) affected
70	20:43:47	CREATE TRIGGER CheckBalance BEFORE UPDATE ON...	0 row(s) affected
71	20:43:47	UPDATE Account SET Balance = Balance - 60000 WHER...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
72	20:44:25	CREATE TRIGGER CheckTransactionAmount BEFORE IN...	0 row(s) affected
73	20:44:25	INSERT INTO Bank_Transaction (Account_No, Transactio...	Error Code: 1644. Transaction amount must be greater than...
74	20:45:09	CREATE TABLE Transaction_Audit (Audit_ID INT PRI...	0 row(s) affected
75	20:45:40	CREATE TRIGGER TransactionAudit AFTER INSERT ON ...	0 row(s) affected
76	20:45:40	INSERT INTO Bank_Transaction (Account_No, Transactio...	1 row(s) affected
77	20:45:40	SELECT * FROM Transaction_Audit LIMIT 0, 1000	1 row(s) returned

20. TRIGGER – AUTOMATICALLY UPDATE ACCOUNT BALANCE

```

DELIMITER //
CREATE TRIGGER UpdateBalanceAfterTransaction
AFTER INSERT ON Bank_Transaction
FOR EACH ROW
BEGIN
    IF NEW.Transaction_Type = 'DEPOSIT' THEN
        UPDATE Account
        SET Balance = Balance + NEW.Amount
        WHERE Account_No = NEW.Account_No;
    
```

```

ELSEIF NEW.Transaction_Type = 'WITHDRAW' THEN
    UPDATE Account
    SET Balance = Balance - NEW.Amount
    WHERE Account_No = NEW.Account_No;

END IF;
END //
DELIMITER ;

```

```

INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'DEPOSIT', 5000);

```

```

SELECT *
FROM Account
WHERE Account_No = 10001;

```

The screenshot displays a database management interface. The top section shows a list of SQL statements with line numbers 693 to 701. The statements include an INSERT into Bank_Transaction and a SELECT from Account. Below the SQL editor, a 'Result Grid' shows the execution results of the SELECT statement, displaying one row for Account_No 10001. The bottom section, 'Action Output', provides a detailed log of database actions, including table creation, trigger creation, and data insertion, with timestamps and row counts.

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	12000.00	Hyderabad

#	Time	Action	Message
72	20:44:25	CREATE TRIGGER CheckTransactionAmount BEFORE IN...	0 row(s) affected
73	20:44:25	INSERT INTO Bank_Transaction (Account_No, Transactio...	Error Code: 1644. Transaction amount must be greater than...
74	20:45:09	CREATE TABLE Transaction_Audit (Audit_ID INT PRI...	0 row(s) affected
75	20:45:40	CREATE TRIGGER TransactionAudit AFTER INSERT ON ...	0 row(s) affected
76	20:45:40	INSERT INTO Bank_Transaction (Account_No, Transactio...	1 row(s) affected
77	20:45:40	SELECT * FROM Transaction_Audit LIMIT 0, 1000	1 row(s) returned
78	20:46:23	CREATE TRIGGER UpdateBalanceAfterTransaction AFTE...	0 row(s) affected
79	20:46:23	INSERT INTO Bank_Transaction (Account_No, Transactio...	1 row(s) affected
80	20:46:23	SELECT * FROM Account WHERE Account_No = 10001 ...	1 row(s) returned

21. TRIGGER – PREVENT WITHDRAWAL ABOVE BALANCE

```

DELIMITER //
CREATE TRIGGER PreventInsufficientWithdrawal
BEFORE INSERT ON Bank_Transaction

```

```
FOR EACH ROW
BEGIN
```

```
    DECLARE CurrentBalance DECIMAL(12,2);
```

```
    SELECT Balance
    INTO CurrentBalance
    FROM Account
    WHERE Account_No = NEW.Account_No;
```

```
    IF NEW.Transaction_Type = 'WITHDRAW'
       AND NEW.Amount > CurrentBalance THEN
```

```
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT =
        'Withdrawal failed: Insufficient balance';
```

```
    END IF;
```

```
END //
DELIMITER ;
```

```
INSERT INTO Bank_Transaction
(Account_No, Transaction_Type, Amount)
VALUES
(10001, 'WITHDRAW', 1000000);
```

The screenshot shows a database management tool interface. On the left, there's a sidebar with 'Administrati...' and 'Information' tabs. The main area is a SQL editor with line numbers 722 to 731. The SQL code is as follows:

```
722
723 END //
724 DELIMITER ;
725
726 INSERT INTO Bank_Transaction
727 (Account_No, Transaction_Type, Amount)
728 VALUES
729 (10001, 'WITHDRAW', 1000000);
730
731
```

Below the editor is an 'Output' window with a tab 'Action Output'. It contains a table with columns: #, Time, Action, Message, and a status icon. The table lists the execution of several SQL statements:

#	Time	Action	Message	Status
74	20:45:09	CREATE TABLE Transaction_Audit (Audit_ID INT PRI...	0 row(s) affected	Success
75	20:45:40	CREATE TRIGGER TransactionAudit AFTER INSERT ON ...	0 row(s) affected	Success
76	20:45:40	INSERT INTO Bank_Transaction (Account_No, Transactio...	1 row(s) affected	Success
77	20:45:40	SELECT * FROM Transaction_Audit LIMIT 0, 1000	1 row(s) returned	Success
78	20:46:23	CREATE TRIGGER UpdateBalanceAfterTransaction AFTE...	0 row(s) affected	Success
79	20:46:23	INSERT INTO Bank_Transaction (Account_No, Transactio...	1 row(s) affected	Success
80	20:46:23	SELECT * FROM Account WHERE Account_No = 10001 ...	1 row(s) returned	Success
81	20:47:02	CREATE TRIGGER PreventInsufficientWithdrawal BEFOR...	0 row(s) affected	Success
82	20:47:02	INSERT INTO Bank_Transaction (Account_No, Transactio...	Error Code: 1644. Withdrawal failed: Insufficient balance	Error

ADVANCED STORED PROCEDURES

22. PROCEDURE – ACCOUNT TRANSFER

DELIMITER //

```
CREATE PROCEDURE TransferMoney(  
    IN SenderAccount INT,  
    IN ReceiverAccount INT,  
    IN TransferAmount DECIMAL(12,2)
```

```
)
```

```
BEGIN
```

```
    DECLARE SenderBalance DECIMAL(12,2);
```

```
    SELECT Balance  
    INTO SenderBalance  
    FROM Account  
    WHERE Account_No = SenderAccount;
```

```
    IF SenderBalance < TransferAmount THEN
```

```
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT =  
        'Transfer failed: Insufficient balance';
```

```
    ELSE
```

```
        UPDATE Account  
        SET Balance = Balance - TransferAmount  
        WHERE Account_No = SenderAccount;
```

```
        UPDATE Account  
        SET Balance = Balance + TransferAmount  
        WHERE Account_No = ReceiverAccount;
```

```
    END IF;
```

```
END //
```

```
DELIMITER ;
```

```
CALL TransferMoney(10001, 10002, 5000);
```

```

SELECT *
FROM Account
WHERE Account_No IN (10001,10002);

```

The screenshot displays a database IDE interface. The top pane shows SQL code with line numbers 761 through 774. The code includes an 'END IF;' statement, a comment 'END //', a 'DELIMITER ;' statement, a 'CALL TransferMoney(10001, 10002, 5000);' statement, and a 'SELECT * FROM Account WHERE Account_No IN (10001,10002);' statement. The bottom pane shows the 'Result Grid' with a table of account data:

Account_No	Customer_ID	Account_Type	Balance	Branch
10001	101	Savings	7000.00	Hyderabad
10002	102	Savings	80000.00	Vijayawada
NULL	NULL	NULL	NULL	NULL

Below the result grid, the 'Action Output' pane shows a log of database actions and their results:

#	Time	Action	Message
77	20:45:40	SELECT * FROM Transaction_Audit LIMIT 0, 1000	1 row(s) returned
78	20:46:23	CREATE TRIGGER UpdateBalanceAfterTransaction AFTE...	0 row(s) affected
79	20:46:23	INSERT INTO Bank_Transaction (Account_No, Transactio...	1 row(s) affected
80	20:46:23	SELECT * FROM Account WHERE Account_No = 10001 ...	1 row(s) returned
81	20:47:02	CREATE TRIGGER PreventInsufficientWithdrawal BEFOR...	0 row(s) affected
82	20:47:02	INSERT INTO Bank_Transaction (Account_No, Transactio...	Error Code: 1644. Withdrawal failed: Insufficient balance
83	20:48:37	CREATE PROCEDURE TransferMoney(IN SenderAcco...	0 row(s) affected
84	20:48:37	CALL TransferMoney(10001, 10002, 5000)	1 row(s) affected
85	20:48:37	SELECT * FROM Account WHERE Account_No IN (1000...	2 row(s) returned

23. PROCEDURE – CUSTOMER LOAN DETAILS

```

DELIMITER //

```

```

CREATE PROCEDURE GetCustomerLoans(
    IN p_Customer_ID INT
)
BEGIN

```

```

    SELECT
        C.Customer_Name,
        L.Loan_ID,

```

```

        L.Loan_Type,
        L.Loan_Amount,
        L.Interest_Rate
FROM Customer C
JOIN Loan L
ON C.Customer_ID = L.Customer_ID
WHERE C.Customer_ID = p_Customer_ID;

END //
DELIMITER ;

```

CALL GetCustomerLoans(101);

The screenshot shows a database management tool interface. The top panel displays SQL code with line numbers 790 to 796. The code includes a WHERE clause, END statement, DELIMITER, and a CALL statement for GetCustomerLoans(101). The middle panel shows the 'Result Grid' with a single row of data for 'Ravi Kumar' with Loan_ID 501, Loan_Type 'Home Loan', Loan_Amount 5000000.00, and Interest_Rate 7.50. The bottom panel shows the 'Action Output' log with a list of actions and their messages, including successful inserts, selects, and procedure calls, as well as an error for a failed withdrawal.

Customer_Name	Loan_ID	Loan_Type	Loan_Amount	Interest_Rate
Ravi Kumar	501	Home Loan	5000000.00	7.50

#	Time	Action	Message
79	20:46:23	INSERT INTO Bank_Transaction (Account_No, Transactio...	1 row(s) affected
80	20:46:23	SELECT * FROM Account WHERE Account_No = 10001 ...	1 row(s) returned
81	20:47:02	CREATE TRIGGER PreventInsufficientWithdrawal BEFOR...	0 row(s) affected
82	20:47:02	INSERT INTO Bank_Transaction (Account_No, Transactio...	Error Code: 1644. Withdrawal failed: Insuffi
83	20:48:37	CREATE PROCEDURE TransferMoney(IN SenderAcco...	0 row(s) affected
84	20:48:37	CALL TransferMoney(10001, 10002, 5000)	1 row(s) affected
85	20:48:37	SELECT * FROM Account WHERE Account_No IN (1000...	2 row(s) returned
86	20:49:09	CREATE PROCEDURE GetCustomerLoans(IN p_Custo...	0 row(s) affected
87	20:49:09	CALL GetCustomerLoans(101)	1 row(s) returned

24. PROCEDURE – HIGH BALANCE ACCOUNTS

```

DELIMITER //
CREATE PROCEDURE HighBalanceAccounts(
    IN MinimumBalance DECIMAL(12,2)
)
BEGIN

    SELECT *
    FROM Account

```

```
WHERE Balance >= MinimumBalance
ORDER BY Balance DESC;
```

```
END //
```

```
DELIMITER ;
```

```
CALL HighBalanceAccounts(50000);
```

The screenshot shows a SQL IDE with a script editor and a result grid. The script editor contains the following SQL code:

```
797 DELIMITER //
798 CREATE PROCEDURE HighBalanceAccounts(
799     IN MinimumBalance DECIMAL(12,2)
800 )
801 BEGIN
802
803     SELECT *
804     FROM Account
805     WHERE Balance >= MinimumBalance
806     ORDER BY Balance DESC;
807
808 END //
809
810 DELIMITER ;
811
812 CALL HighBalanceAccounts(50000);
813
```

The result grid shows the output of the procedure call, displaying a list of accounts with balances greater than or equal to 50,000. The result grid has columns: Account_No, Customer_ID, Account_Type, Balance, and Branch. The data is as follows:

Account_No	Customer_ID	Account_Type	Balance	Branch
10007	107	Current	150000.00	Mumbai
10003	103	Current	120000.00	Bangalore
10010	110	Current	110000.00	Vijayawada
10005	105	Current	90000.00	Hyderabad
10009	109	Savings	85000.00	Hyderabad
10002	102	Savings	80000.00	Vijayawada
10006	106	Savings	65000.00	Delhi

25. PROCEDURE WITH IN, OUT PARAMETERS

```
DELIMITER //
```

```
CREATE PROCEDURE GetBalance(
    IN p_Account_No INT,
    OUT p_Balance DECIMAL(12,2)
)
BEGIN
```

```
SELECT Balance
INTO p_Balance
FROM Account
WHERE Account_No = p_Account_No;
```

```
END //
```

```
DELIMITER ;
```

```
CALL GetBalance(10001, @CurrentBalance);
```

```
SELECT @CurrentBalance;
```

```
815 DELIMITER //
816
817 CREATE PROCEDURE GetBalance(
818     IN p_Account_No INT,
819     OUT p_Balance DECIMAL(12,2)
820 )
821 BEGIN
822
823     SELECT Balance
824     INTO p_Balance
825     FROM Account
826     WHERE Account_No = p_Account_No;
827
828 END //
829
830 DELIMITER ;
831
832 CALL GetBalance(10001, @CurrentBalance);
833
834 SELECT @CurrentBalance;
835
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

@CurrentBalance
7000.00

TRIGGER MANAGEMENT COMMANDS

```
SHOW TRIGGERS;
```

836 • **SHOW TRIGGERS;**

837

Result Grid Filter Rows: Export: Wrap Cell Content:

	Trigger	Event	Table	Statement
▶	CheckBalance	UPDATE	account	BEGIN IF NEW.Balance < 0 THEN
	CheckTransactionAmount	INSERT	bank_transaction	BEGIN IF NEW.Amount <= 0 THEN
	PreventInsufficientWithdrawal	INSERT	bank_transaction	BEGIN DECLARE CurrentBalance DE
	TransactionAudit	INSERT	bank_transaction	BEGIN INSERT INTO Transaction_At
	UpdateBalanceAfterTransaction	INSERT	bank_transaction	BEGIN IF NEW.Transaction_Type =

SHOW CREATE TRIGGER CheckBalance;

837 • **SHOW CREATE TRIGGER CheckBalance;**

Result Grid Filter Rows: Export: Wrap Cell Content:

	Trigger	sql_mode	SQL Original Statement
▶	CheckBalance	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	CREATE DEFINER='root'@'localhost

DROP TRIGGER CheckBalance;

839 • **DROP TRIGGER CheckBalance;**

840

Output

Action Output

#	Time	Action	Message
95	20:51:24	SHOW CREATE TRIGGER CheckBalance	1 row(s) returned
96	20:51:41	DROP TRIGGER CheckBalance	0 row(s) affected

PROCEDURE MANAGEMENT COMMANDS

===== SHOW

PROCEDURE STATUS

WHERE Db = 'BankDB';

0-40

841 • SHOW PROCEDURE STATUS

842 WHERE Db = 'BankDB';

843

Result Grid						
		Filter Rows:	Export:		Wrap Cell Content:	
	Db	Name	Type	Definer	Modified	Created
	bankdb	GetCustomerAccounts	PROCEDURE	root@localhost	2026-08-13 20:41:24	2026-08-13 20:41:24
	bankdb	GetCustomerLoans	PROCEDURE	root@localhost	2026-08-13 20:49:09	2026-08-13 20:49:09
	bankdb	HighBalanceAccounts	PROCEDURE	root@localhost	2026-08-13 20:49:41	2026-08-13 20:49:41
	bankdb	TransferMoney	PROCEDURE	root@localhost	2026-08-13 20:48:37	2026-08-13 20:48:37
	bankdb	WithdrawMoney	PROCEDURE	root@localhost	2026-08-13 20:43:19	2026-08-13 20:43:19